Household Services Application

Modern Application Development I
September, 2024 Term

❖ Student Details

Name – Sudip Dey
Roll no. – 23f3004246
Email – 23f3004246@ds.study.iitm.ac.in

❖ Project Description

It is a multi-user app (requires one admin and other service professionals/ customers) which acts as platform for providing comprehensive home servicing and solutions.

❖ How I approached the problem statement

- **Understanding the Requirements:**
  - I carefully analyzed the problem statement, which involved creating a multi-user household service application with roles for customers, service professionals, and admins.
  - The application required functionalities like service booking, dashboard views, profile management, and role-specific workflows.

- **Designing the Architecture:**
  - **Customer**: Can book services, view requests, update profiles, and rate completed services.
  - **Service Professional**: Can view and manage service requests based on availability and location.
  - **Admin**: Manages users, services, and reviews reports.

- **Implementing Features:**
  - Ensured proper relationships, like many-to-one connections between service requests and users.
  - Used Flask's routing mechanism (@app.route) to create endpoints for all key functionalities.
  - Designed role-specific dashboards using role validation via session management.
  - Implemented request handling for customers to book services and professionals to accept/reject requests.
  - Added time management logic to restrict professionals from accepting requests exceeding 12 hours per day.
  - Enabled secure user authentication using session tokens.
  - Built profile update forms for all roles with fields specific to their needs.
  - Created a file upload mechanism for service professionals to upload CVs securely, ensuring the use of proper filenames and extensions.

- **Frontend Integration:**
  - Developed role-specific dashboards and forms with Bootstrap for a responsive user interface.
  - Integrated templates with dynamic data injection using Flask's render_template.
  - Ensured that invalid inputs, such as duplicate email addresses or incorrect file formats, were handled gracefully using flash messages.

- **Deployment and Scalability:**
  - Ensured modular code design for scalability and maintainability.
  - Prepared the application to handle future enhancements, such as adding more roles or extending functionality.

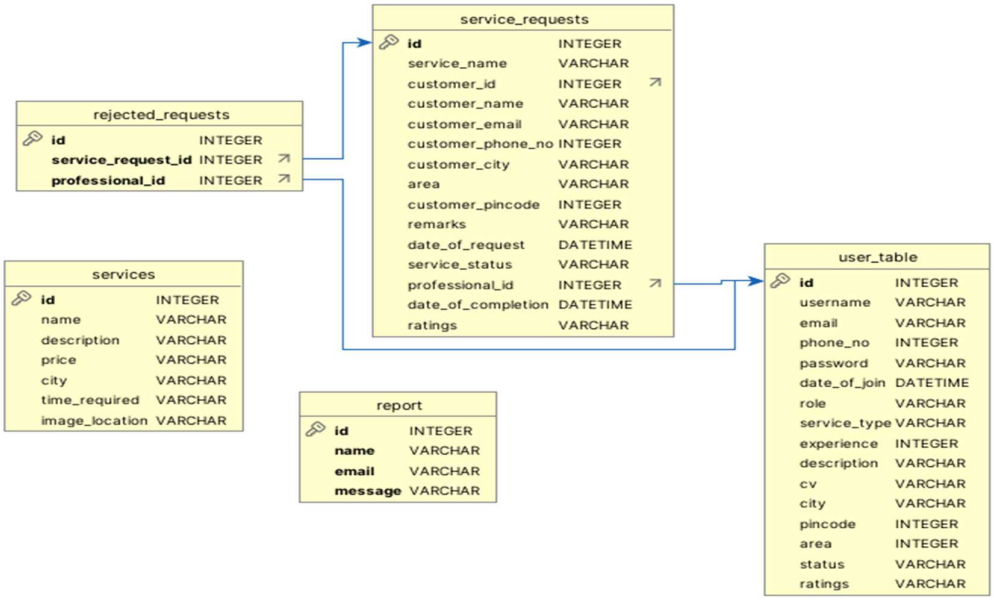
The approach provided a robust, user-friendly application with clearly defined workflows for each role, ensuring seamless interaction between customers, service professionals, and admins.

❖ Technologies Used

- **Flask:** Backend framework for building the web application.
- **SQL Alchemy:** ORM (Object-Relational Mapping) tool for database interactions.
- **SQLite:** Database management system for storing application data.
- **HTML/CSS/JavaScript:** Frontend technologies for user interface design and interactivity.
- **Datetime:** Python library for handling date and time operations.
- **Jinja2:** Template engine for rendering dynamic HTML content.
- **Werkzeug:** Ensures uploaded file names are secure (e.g., removes special characters).
- **ChartJS:** User for creating different types of charts on the admin dashboard.

❖ Db Schema Design

The database schema encompasses tables for user, services, service request, reject requests, report. These entities are interconnected to track various user activities within the system. Each table contains essential fields such as user details, services, service request, reject requests, user ratings and feedback data. This structure supports comprehensive tracking and management of activities related to services, service request, reject requests and user engagements within the application.



❖ Project video

Drive: *https://drive.google.com/file/d/1fnPxtrkpeH9_BkrFvPKMyxE0uuHQRVHG/view?usp=sharing*

Youtube: *https://youtu.be/E53pkOan4so*